# Market Basket Analysis And Clustering

## From Shopping Carts to Census Data — and Beyond

Based on: Szymkowiak et al. (2018) · Raeder & Chawla (2011)

Mehreen Ali Gillani
Izza Khan
Sachi kapoor

Predictive Analytics - DATA624
Professor Nieman Jeffery

# Presentation Outline


**01** **What is Market Basket Analysis?**
Core concepts & origins

**02** **Key Metrics**
Support · Confidence · Lift

**03** **The Apriori Algorithm**
How rules are discovered

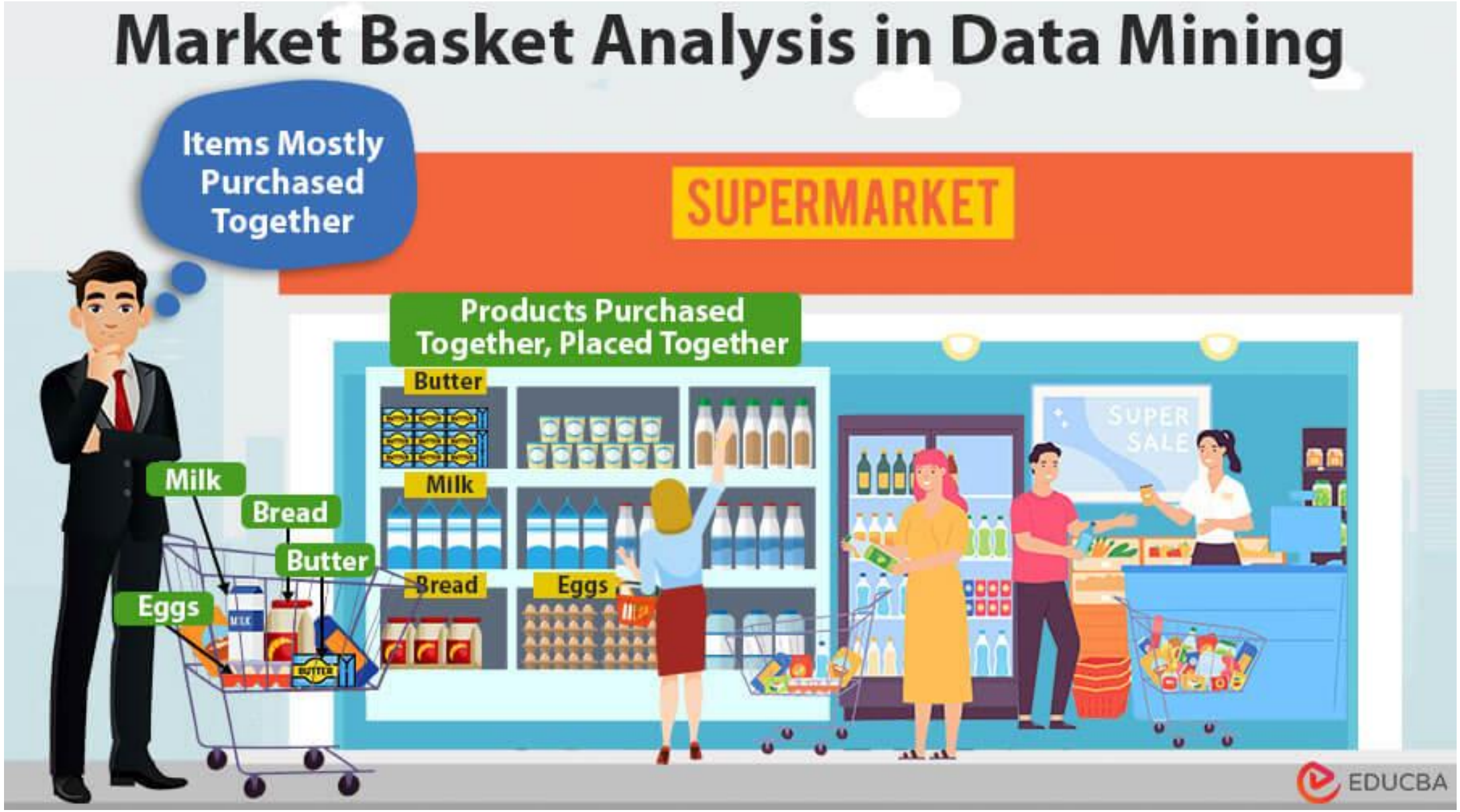
**04** **FP Growth**
Retail & e-commerce

**05** **Comparison between Apriori and FP Growth**
Same output. Different path

**06** **Network-Based MBA**
Product networks & communities (Raeder & Chawla)

**07** **Clustering**
Applying Clustering to Market Basket Analysis

**08** **Summary & Takeaways**
Key learnings


Market Basket Analysis in Data Mining
Items Mostly Purchased Together
SUPERMARKET
Products Purchased Together, Placed Together
Butter
Milk
Bread
Eggs
Milk
Bread
Butter
Eggs
SUPER SALE
EDUCBA

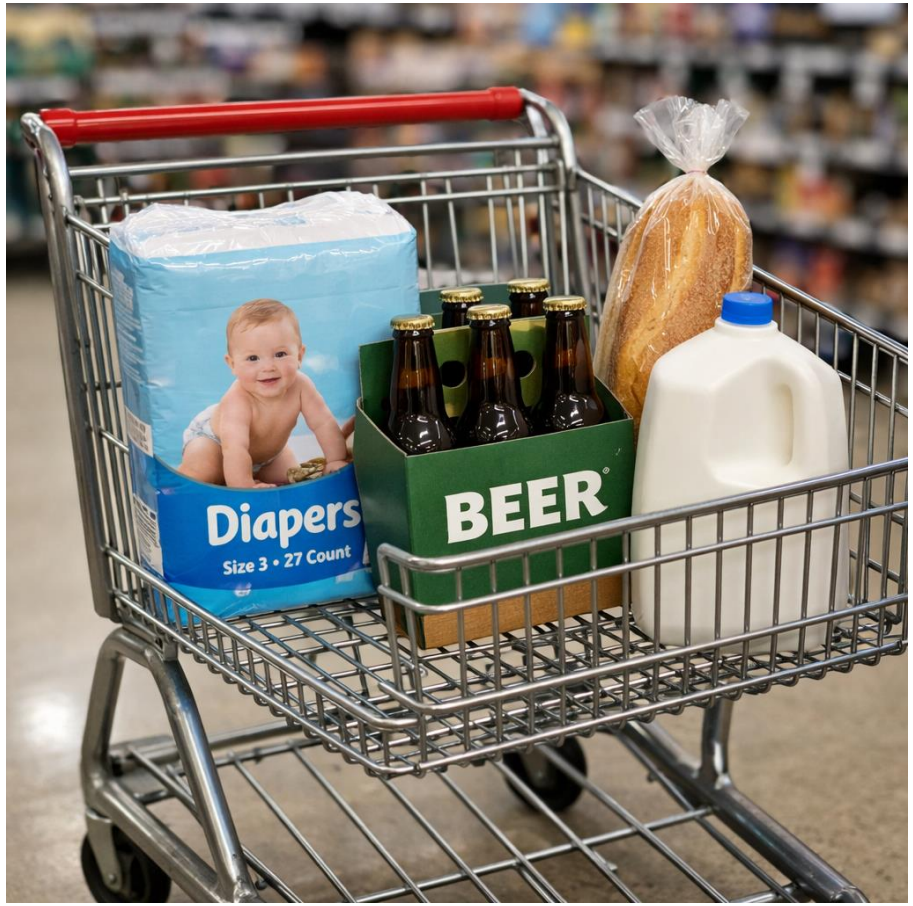
# What is Market Basket Analysis (MBA)?

MBA discovers **co-occurrence** relationships among items in transaction data.

*It answers: "When a customer buys X, how likely are they to also buy Y?"*

**Origin Story (Raeder & Chawla 2011)**

- Retailers issue loyalty cards
- Collect massive transaction databases.
- **Goal**: Find actionable patterns for product placement, promotions, and cross-selling.



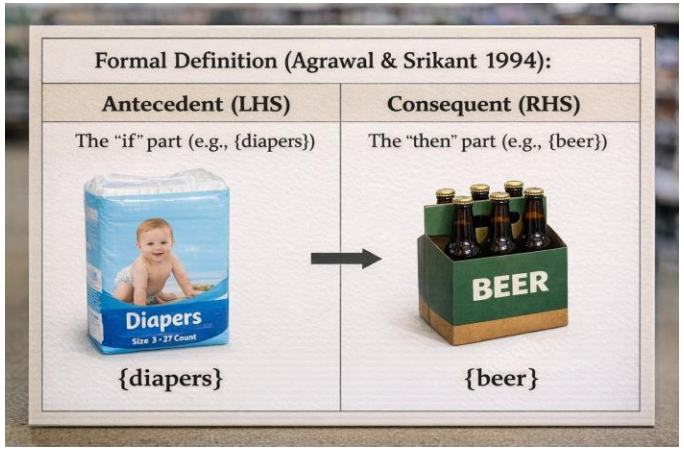

## Key Distinction (Szymkowiak et al. 2018)

| Traditional View | Extended View |
|---|---|
| Products in a physical basket | Any set of categorical attributes |
| Grocery, retail | Demographics, services, web pages |

# Association Rules — The IF-THEN Statement

- The Building Block: {A} → {B} *"IF customer buys A, THEN they also buy B"*

| Type | Example | Why It Occurs | Business / Analytical Value |
|---|---|---|---|
| Trivial | Shirt → Tie | Reflects obvious domain knowledge everyone already knows | No value – confirm it exists, then deprioritize |
| Useful | Diapers → Beer | Hidden behavioral pattern not predictable from intuition | Goldmine – actionable for placement, promotion, targeting |
| Inexplicable | Toothpaste → Guitar strings | Statistically valid co-occurrence with no causal story | Use with caution – may reflect data artifact or confound |
| Spurious | Product A → Product B (low support, high lift) | Random co-occurrence in small samples – not reproducible | Danger – looks interesting, disappears with more data |




The Key Question

*"Is this rule surprising AND profitable?"*

# The Three Core Metrics

## Support

**$P(A \cap B)$**

How often do A and B appear together?
*How Frequent is the Combination?*

$= n(A \cap B) / N$
Proportion of transactions containing both A and B.

100,000 total transactions.
5,000 contain both {coffee} and {cream}
Support = 5,000 / 100,000 = 0.05 (5%)

**High support = frequent pair**

## Confidence

**$P(B \mid A)$**

Given A was bought, how often is B also bought?
*How Reliable is the Rule?*

$= n(A \cap B) / n(A)$
Conditional probability — reliability of the rule.

{coffee, cream} appears in 5,000 transactions
{coffee} appears in 20,000 transactions.
Confidence = 5,000 / 20,000 = 0.25 (25%)

**High confidence = reliable rule**

## Lift

**Conf / P(B)**

Does buying A actually increase the chance of B?
*How Much Better than Chance?*

Lift > 1: positive association
Lift = 1: independent
Lift < 1: negative association

**Lift > 1 = genuine association**

# Apriori Algorithm – The Engine Behind Market Basket Analysis

Find frequent itemsets and generate association rules from transaction data

## 1 Input: Transaction Data

| TID | Items |
|---|---|
| T1 | {Milk, Bread, Eggs} |
| T2 | {Milk, Bread} |
| T3 | {Milk, Eggs} |
| T4 | {Bread, Eggs} |
| T5 | {Milk, Bread, Eggs} |

## 2 Set Minimum Support

min_support = 2

(Example)

## 3 Find Frequent 1-Itemsets ($L_1$)

| Item | Support Count | Frequent? |
|---|---|---|
| Milk | 4 | ✓ |
| Bread | 4 | ✓ |
| Eggs | 4 | ✓ |

$L_1$ = {Milk}, {Bread}, {Eggs}

## 4 Generate Candidate 2-Itemsets ($C_2$) (From $L_1$)

$C_2$

{Milk, Bread}

{Milk, Eggs}

{Bread, Eggs}

## 5 Scan Database & Count Support

| Itemset | Support Count | Frequent? |
|---|---|---|
| {Milk, Bread} | 3 | ✓ |
| {Milk, Eggs} | 3 | ✓ |
| {Bread, Eggs} | 3 | ✓ |

$L_2$ = {{Milk, Bread}, {Milk, Eggs}, {Bread, Eggs}}

## 6 Generate Candidate k-Itemsets ($C_k$) from $L_{k-1}$ & Prune

**Join:** Combine itemsets from $L_{k-1}$

**Prune:** Remove candidates with any (k-1)-subset not in $L_{k-1}$

$C_3$ (After Join)

{Milk, Bread, Eggs}

**Prune Step**

Keep if all (k-1)-subsets are in $L_2$

✓ All subsets are frequent

## 7 Scan Database & Count Support

| Itemset | Support Count | Frequent? |
|---|---|---|
| {Milk, Bread, Eggs} | 2 | ✓ |

$L_3$ = {{Milk, Bread, Eggs}}

**Iterate**

Repeat steps 4–7 to generate $C_k$ and $L_k$ until $C_k$ is empty

## 8 All Frequent Itemsets

$L_1$ = {Milk}, {Bread}, {Eggs}

$L_2$ = {{Milk, Bread}, {Milk, Eggs}, {Bread, Eggs}}

$L_3$ = {{Milk, Bread, Eggs}}

## 9 Generate Association Rules

Milk, Bread → Eggs (support=2, confidence=0.67)

Milk, Eggs → Bread (support=2, confidence=0.67)

Bread, Eggs → Milk (support=2, confidence=0.67)

...

**Use Rules for:**

- ✓ Cross-selling
- ✓ Recommendations
- ✓ Store Layout
- ✓ Promotions

**Key Idea: Apriori Property**

If an itemset is frequent, then all of its subsets must be frequent.
This property allows Apriori to prune the search space and improve efficiency.

# Apriori Algorithm

**The Problem** Brute-force search over all possible itemset combinations is computationally explosive — a store with 1,000 products has $2^{1000}$ possible subsets.

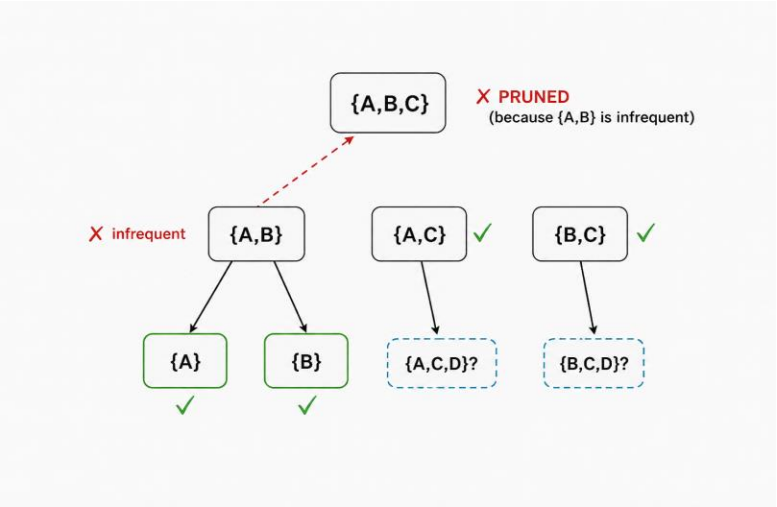
**The Apriori Property (core principle — worth a callout box)**

Every non-empty subset of a frequent itemset must also be frequent.

Its contrapositive is the key to pruning:

If {A, B} is infrequent → {A, B, C} can never be frequent — skip it.

**Why "Apriori"?** The name reflects prior knowledge — the algorithm applies what it already knows about smaller itemsets to prune the search space before ever counting larger ones.




**Complexity Trade-off**

| Pro | Con |
|---|---|
| Drastically faster than brute force | Expensive when data is dense or support is very low |
| Guaranteed to find all frequent itemsets | Multiple passes over the full database required |

# FP-Growth – Scaling Market Basket Analysis to Big Data

## 1 Input: Transaction Database

| TID | Items |
|---|---|
| T1 | {Milk, Bread, Eggs} |
| T2 | {Milk, Bread} |
| T3 | {Milk, Eggs, Beer} |
| T4 | {Bread, Eggs, Beer} |
| T5 | {Milk, Bread, Beer} |
| T6 | {Bread, Eggs} |

**Min Support = 2**

## 2 Scan 1: Build Frequency Table (1-Itemsets)

| Item | Support Count |
|---|---|
| Milk | 4 |
| Bread | 5 |
| Eggs | 4 |
| Beer | 3 |

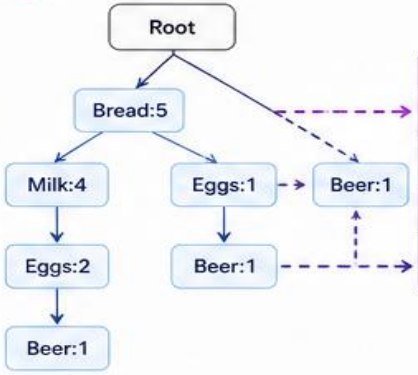
Remove infrequent items (Support < 2) ✓

## 3 Order Items by Frequency (Descending)

| | |
|---|---|
| Bread | (5) |
| Milk | (4) |
| Eggs | (4) |
| Beer | (3) |

This order will be used to build the FP-Tree ✓

## 4 Scan 2: Build FP-Tree




**Header Table (Linked List)**

| Item | Node Link |
|---|---|
| Bread | → |
| Milk | → |
| Eggs | → |
| Beer | → |

## 5 FP-Tree (Compact Representation)

- No candidate generation
- No repeated database scan
- Highly compressed structure
- Efficient for large datasets

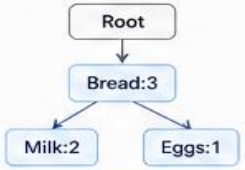
## 6 Mine Frequent Itemsets Using Conditional Pattern Base

### For Beer

**Conditional Pattern Base**

| Path | Count |
|---|---|
| Bread, Milk, Eggs | 1 |
| Bread, Milk | 1 |
| Bread, Eggs | 1 |

**Conditional FP-Tree**



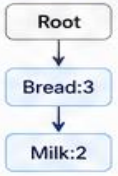

**Frequent Itemsets:** {Beer}, {Milk, Beer}, {Eggs, Beer}, {Bread, Beer}, {Milk, Eggs, Beer}, {Bread, Milk, Beer}, {Bread, Eggs, Beer}, {Bread, Milk, Eggs, Beer}

### For Eggs

**Conditional Pattern Base**

| Path | Count |
|---|---|
| Bread, Milk | 2 |
| Bread | 1 |

**Conditional FP-Tree**




**Frequent Itemsets:** {Eggs}, {Milk, Eggs}, {Bread, Eggs}, {Bread, Milk, Eggs}

### For Milk

**Conditional Pattern Base**

| Path | Count |
|---|---|
| Bread | 4 |

**Conditional FP-Tree**

**Frequent Itemsets:** {Milk}, {Bread, Milk}

### For Bread

**Conditional Pattern Base**

| Path | Count |
|---|---|
| Ø | 5 |

**Conditional FP-Tree**

**Frequent Itemsets:** {Bread}

## 7 All Frequent Itemsets

**1-itemsets:** {Bread}, {Milk}, {Eggs}, {Beer}

**2-itemsets:** {Bread, Milk}, {Bread, Eggs}, {Bread, Beer}, {Milk, Eggs}, {Milk, Beer}, {Eggs, Beer}

**3-itemsets:** {Bread, Milk, Eggs}, {Bread, Milk, Beer}, {Bread, Eggs, Beer}, {Milk, Eggs, Beer}

**4-itemset:** {Bread, Milk, Eggs, Beer}

## 8 Generate Association Rules

| | |
|---|---|
| Bread → Milk | (support=4, confidence=0.80) |
| Milk → Eggs | (support=3, confidence=0.75) |
| Bread, Milk → Beer | (support=2, confidence=0.50) |
| Eggs → Beer | (support=2, confidence=0.50) |
| | ... |

## 9 Business Value

- ✓ Discover cross-selling opportunities
- ✓ Improve recommendations
- ✓ Optimize store layout
- ✓ Increase customer satisfaction
- ✓ Drive higher sales

## Key Advantage of FP-Growth

No candidate generation & no repeated database scan – makes it highly scalable for big data!

# FP-Growth – Scaling Market Basket Analysis to Big Data

**The Bottleneck Apriori** Hits At scale: millions of transactions, thousands of products

**Apriori stalls on two problems:**

- **Candidate generation explodes:** exponentially many itemsets to check
- **Repeated database scans:** one full pass per itemset size level (k=1, 2, 3...)

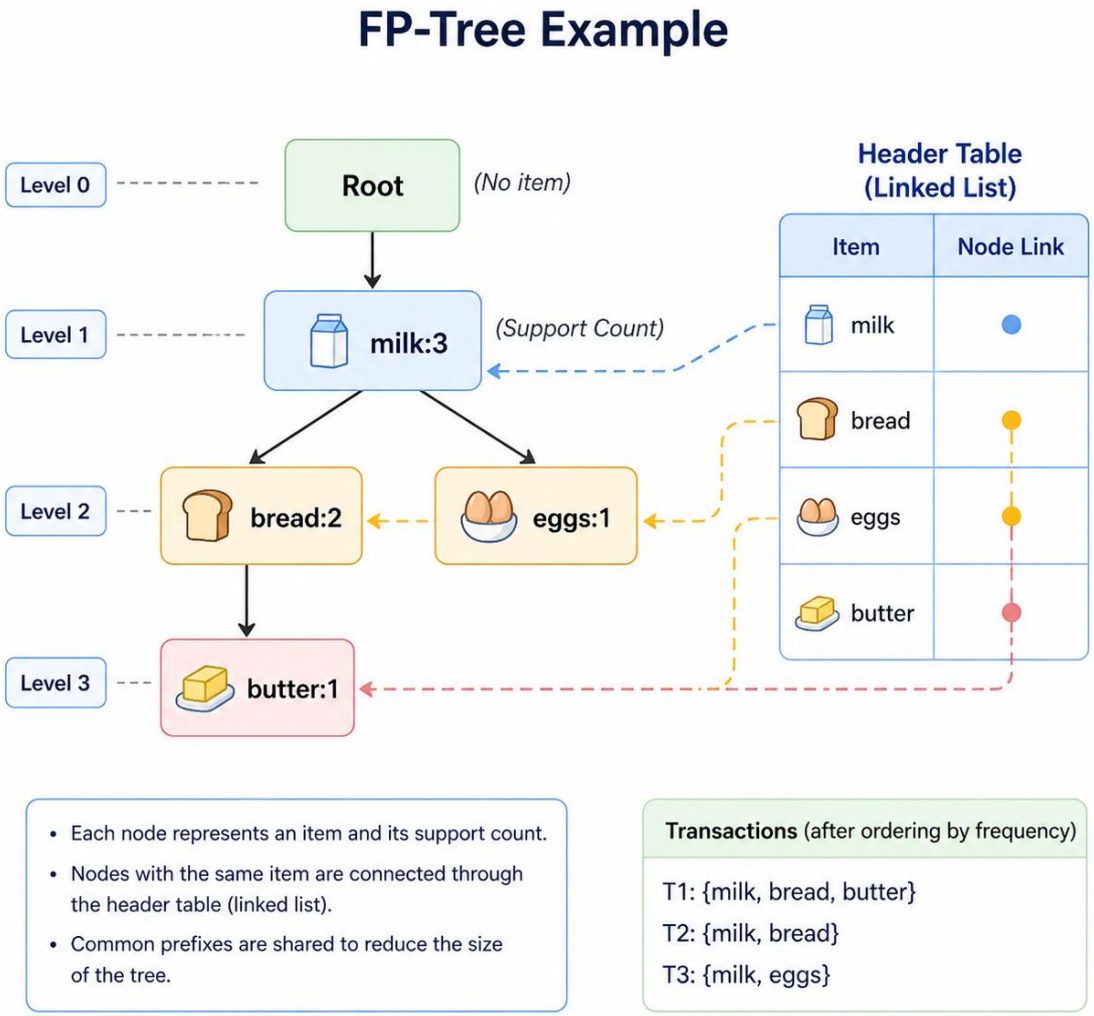
## FP-Growth's Two-Part Solution

**Part 1 – Compress the database into an FP-Tree**

Scan the database *twice only* – once to find frequent 1-itemsets, once to build the tree. Every transaction path is stored as a branch; shared prefixes are merged, so the tree is far smaller than the raw data.

**Part 2 – Mine the tree recursively, no candidates needed**

For each frequent item, extract its *conditional pattern base* (all paths leading to it), build a smaller *conditional FP-Tree*, and repeat. Frequent itemsets emerge directly from the tree structure



# Apriori vs. FP-Growth

| Aspect | Apriori | FP-Growth |
|---|---|---|
| Database scans | Multiple (often 10+) | Two only |
| Implementation complexity | Simple, easy to code | More complex, requires tree structure |
| Handles dense data? | Poor (combinatorial explosion) | Good (compression still effective) |
| Candidate generation | Yes (exponential) | No |
| Memory | Low per scan | Higher (stores tree) |
| Speed on large data | Slow | 2–20x faster |
| When to use | Teaching, small data | Industrial use, lage data, Millions of transactions |

# Code Implementation

python

```python
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, fpgrowth

# Create dataset
dataset = [
    ['Bread', 'Milk'],
    ['Bread', 'Diaper', 'Beer', 'Eggs'],
    ['Milk', 'Diaper', 'Beer', 'Coke'],
    ['Bread', 'Milk', 'Diaper', 'Beer'],
    ['Bread', 'Milk', 'Diaper', 'Coke'],
    ['Bread', 'Milk'],
    ['Diaper', 'Beer'],
    ['Bread', 'Diaper', 'Beer'],
    ['Bread', 'Milk', 'Eggs'],
    ['Milk', 'Diaper', 'Beer', 'Eggs']
]

# One-hot encode
te = TransactionEncoder()
df = pd.DataFrame(te.fit_transform(dataset), columns=te.columns_)

# Apriori
frequent_apriori = apriori(df, min_support=0.2, use_colnames=True)
rules_apriori = association_rules(frequent_apriori, metric="lift", min_threshold=1)

# FP-Growth
frequent_fpgrowth = fpgrowth(df, min_support=0.2, use_colnames=True)
rules_fpgrowth = association_rules(frequent_fpgrowth, metric="lift", min_threshold=1)
```

r

```r
library(arules)

# Create transactions
trans_list <- list(
    c("Bread", "Milk"),
    c("Bread", "Diaper", "Beer", "Eggs"),
    c("Milk", "Diaper", "Beer", "Coke"),
    c("Bread", "Milk", "Diaper", "Beer"),
    c("Bread", "Milk", "Diaper", "Coke"),
    c("Bread", "Milk"),
    c("Diaper", "Beer"),
    c("Bread", "Diaper", "Beer"),
    c("Bread", "Milk", "Eggs"),
    c("Milk", "Diaper", "Beer", "Eggs")
)

trans <- as(trans_list, "transactions")

# Apriori
rules_apriori <- apriori(trans,
    parameter = list(support = 0.2, confidence = 0.6))

# FP-Growth
rules_fpgrowth <- fim4r(trans,
    method = "fpgrowth",
    support = 0.2,
    confidence = 0.6)
```

# The curse of dimensionality — Too Many Rules

*For every interesting rule, hundreds of boring ones*

**Table 1** High, low, and mean rank and standard deviation of ranks for the top 10 rules by average rank among the 21 interestingness measures in Tan et al. (2004)

| Rule | High | Low | Mean | SD |
|---|---|---|---|---|
| CREAM_CHEESE → BAGEL | 1 | 128 | 18.07 | 33.79 |
| Cake Mix[a] → Frosting | 3 | 65.5 | 21.85 | 19.61 |
| VAULT_SODA → VAULT_ZERO | 6 | 71 | 24.95 | 15.77 |
| YORK_MINT_PATTIES, DIET_COKE_20_OZ → NEWSPAPER_CHICAGO_TR | 2 | 96 | 28.05 | 25.67 |
| NEWSPAPER_CHICAGO_TR, DIET_COKE_20_OZ → YORK_MINT_PATTIES | 8 | 96 | 28.85 | 22.90 |
| BAGEL → CREAM_CHEESE | 1 | 129 | 31.37 | 36.32 |
| CREAM_CHEESE, COFFEE_12_OZ → BAGEL | 1 | 133 | 32.02 | 42.90 |
| NYQUIL → DAYQUIL | 1 | 118.5 | 33.35 | 33.92 |
| VAULT_ZERO → VAULT_SODA | 16 | 70 | 33.40 | 13.41 |
| Frosting → Cake Mix | 3 | 69 | 34.37 | 43.42 |

*Table 1 shows information about the top ten rules by average rank. Association rules at 0.01% support and 10% confidence and ranked them according to each measure given by Tan et al. (2004).*

## Three Attempted Solutions – All Insufficient

| Method | Result |
|---|---|
| Maximal/closed itemsets | Pruned 13 of 168 rules |
| Hyperclique patterns | Zero rules of size >2 |
| Interestingness measures | 20+ measures → disagree completely |

**The Core Insight:**

*Mining rules is fast. Knowing which rules matter is not.*

# Beyond Traditional MBA – Why Networks? Moving from Rules to Networks

**Product network construction (Raeder & Chawla, Sec 3):**

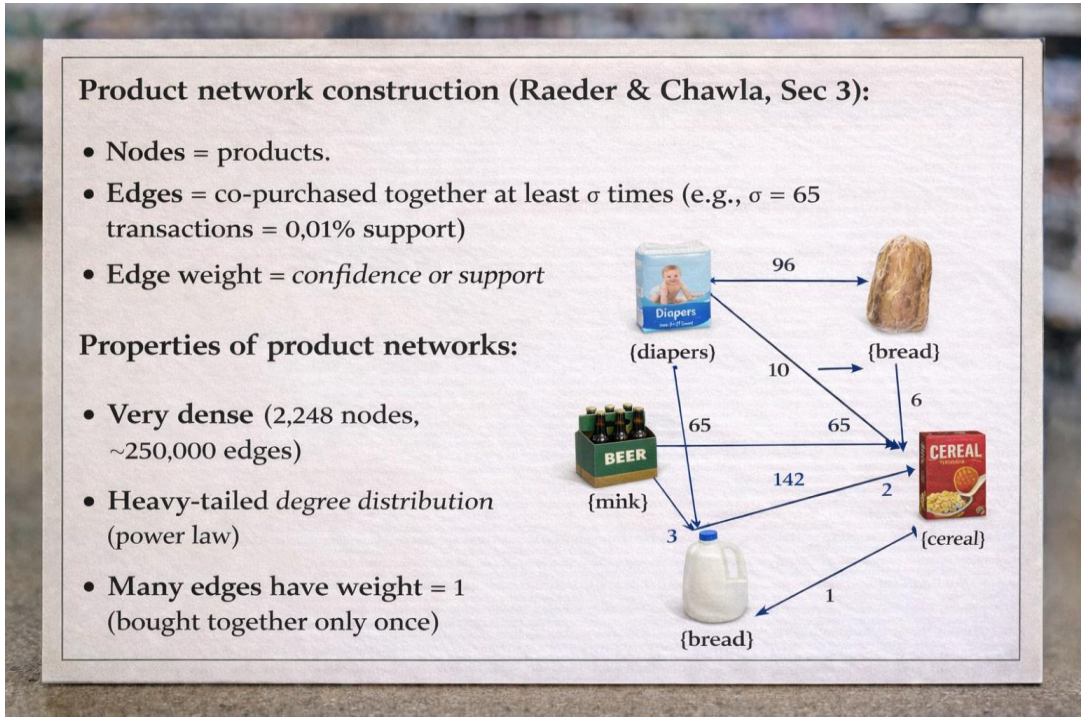
- Nodes = products.
- Edges = co-purchased together at least σ times (e.g., σ = 65 transactions = 0.01% support).
- Edge weight = confidence or support.

**Properties of product networks:**

- Very dense (2,248 nodes, ~250,000 edges).
- Heavy-tailed degree distribution (power law).
- Many edges have weight = 1 (bought together only once).

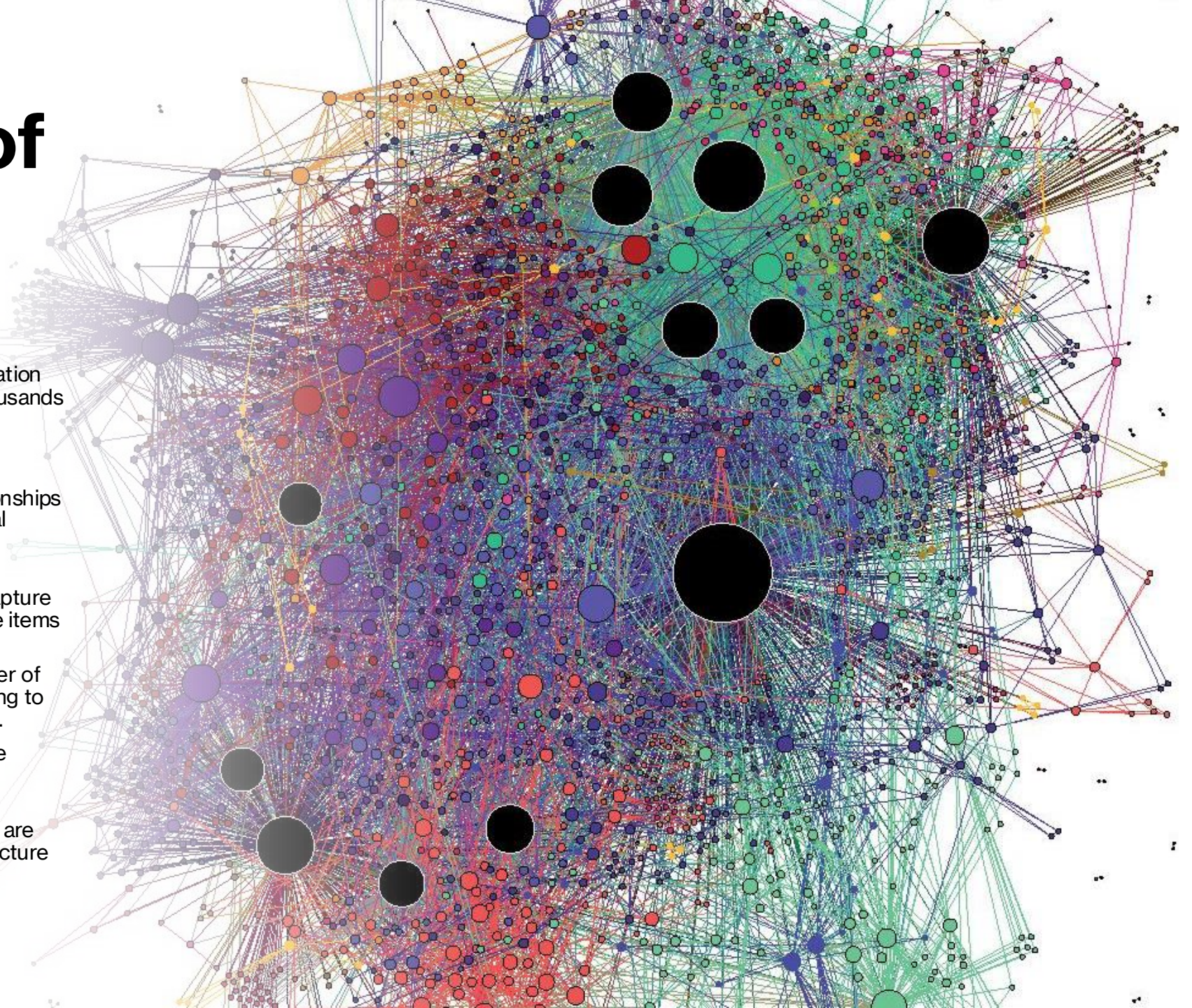
**Why communities?**

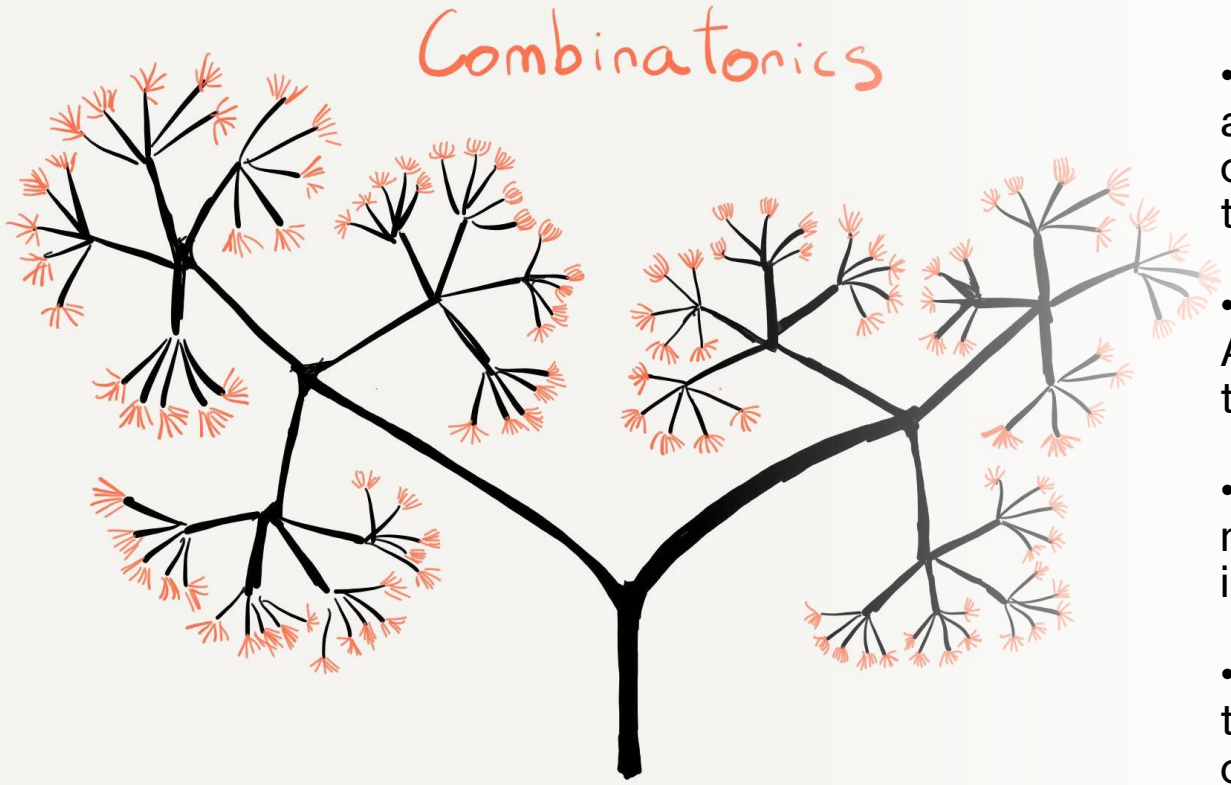
"Communities of products that are more strongly connected internally than externally can reveal expressive relationships without the explosion of rules."



# Limitations of Association Rule Mining

- In large-scale transactional datasets, association rule mining often generates hundreds to thousands of rules, many of which provide little to no actionable insight.
- A significant proportion of these rules are redundant or trivial, reflecting obvious relationships rather than uncovering meaningful behavioral patterns.
- Traditional rule mining is inherently limited to pairwise relationships (A → B) and fails to capture higher-order dependencies involving multiple items simultaneously.
- As the number of items increases, the number of possible itemsets grows exponentially, leading to what is known as the rule explosion problem.
- This results in reduced interpretability, where analysts struggle to distinguish statistically significant relationships from noise.
- Consequently, rule-based approaches alone are insufficient for understanding the global structure of purchasing behavior in complex datasets.






# The Rule Explosion Problem

- In large-scale transactional datasets, association The number of possible item combinations increases combinatorially with the number of products in a dataset.

- Even with pruning techniques (such as Apriori), large datasets can still produce thousands of candidate rules.

- Many rules may have high statistical measures (e.g., lift) but lack practical interpretability or causal explanation.

- Existing filtering techniques (e.g., support thresholds, confidence thresholds) often fail to consistently identify meaningful rules.

- This creates a gap between computational output and human interpretability, limiting the usefulness of rule-based insights in real-world decision-making.

# From Rule-Based Analysis to Network-Based Modeling

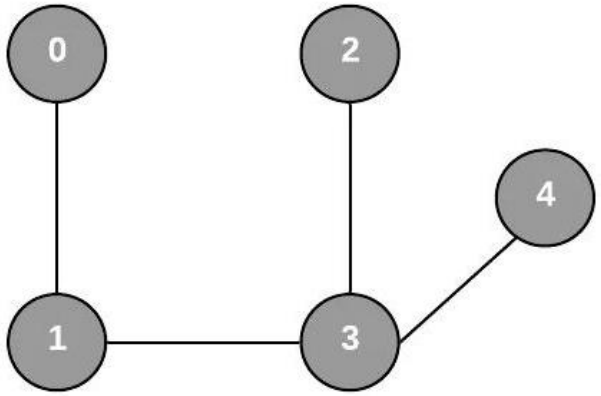
- To overcome these limitations, we transition from analyzing individual association rules to modeling data as a network of interconnected relationships.

- In this framework, the dataset is represented as a graph structure, where all interactions are analyzed collectively rather than independently.

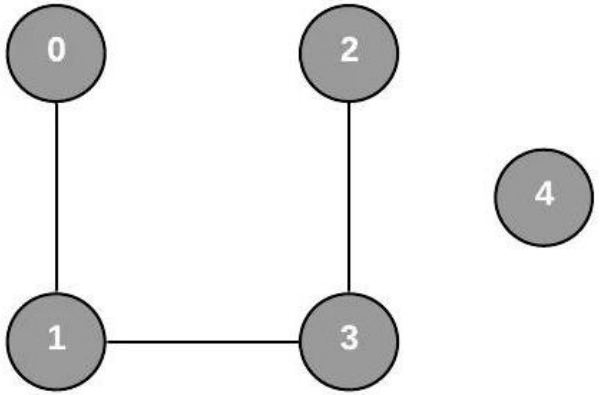
- This enables a shift from a local perspective (isolated rules) to a global perspective (entire system of relationships).

- Network-based approaches allow us to uncover emergent structures, which are patterns that arise from the interaction of multiple components.

- This paradigm aligns with modern data science techniques that emphasize structure, connectivity, and pattern discovery in complex systems.




Connected Graph




Disconnected Graph

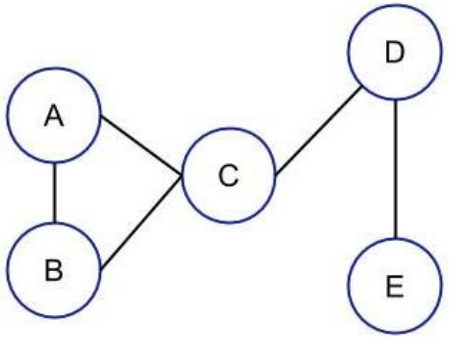



**Figure:** Connected Graph

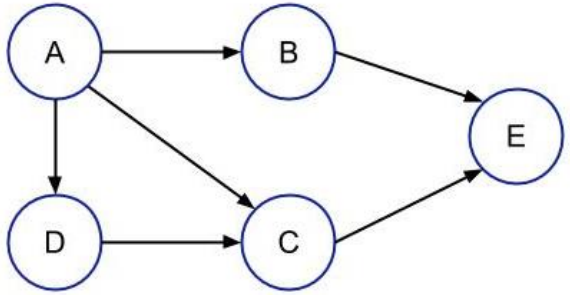
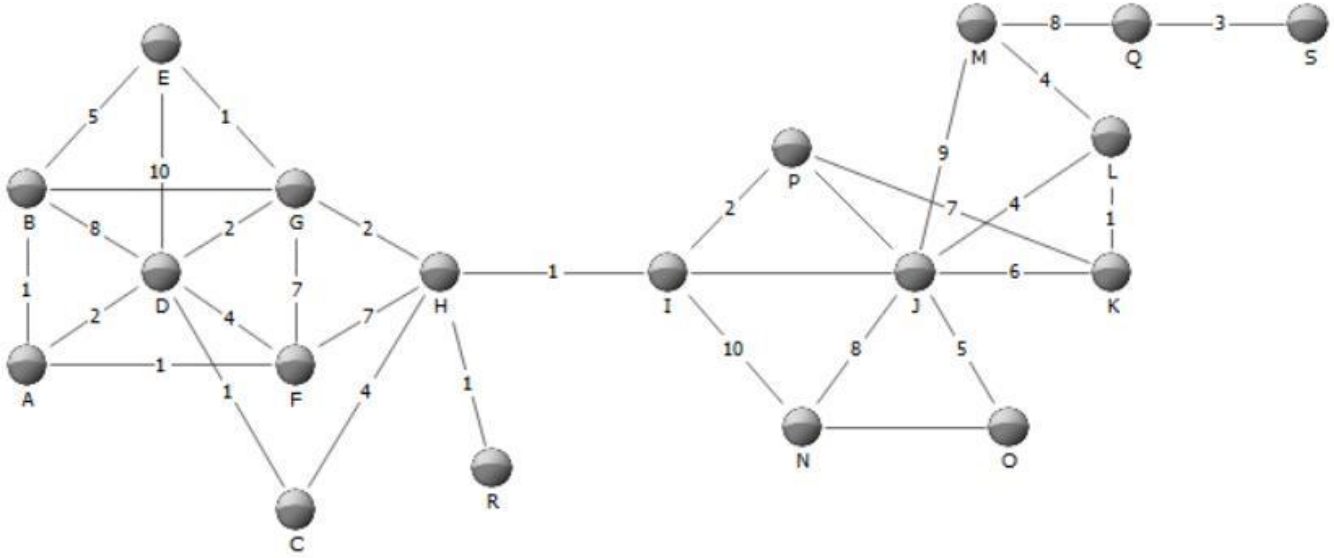


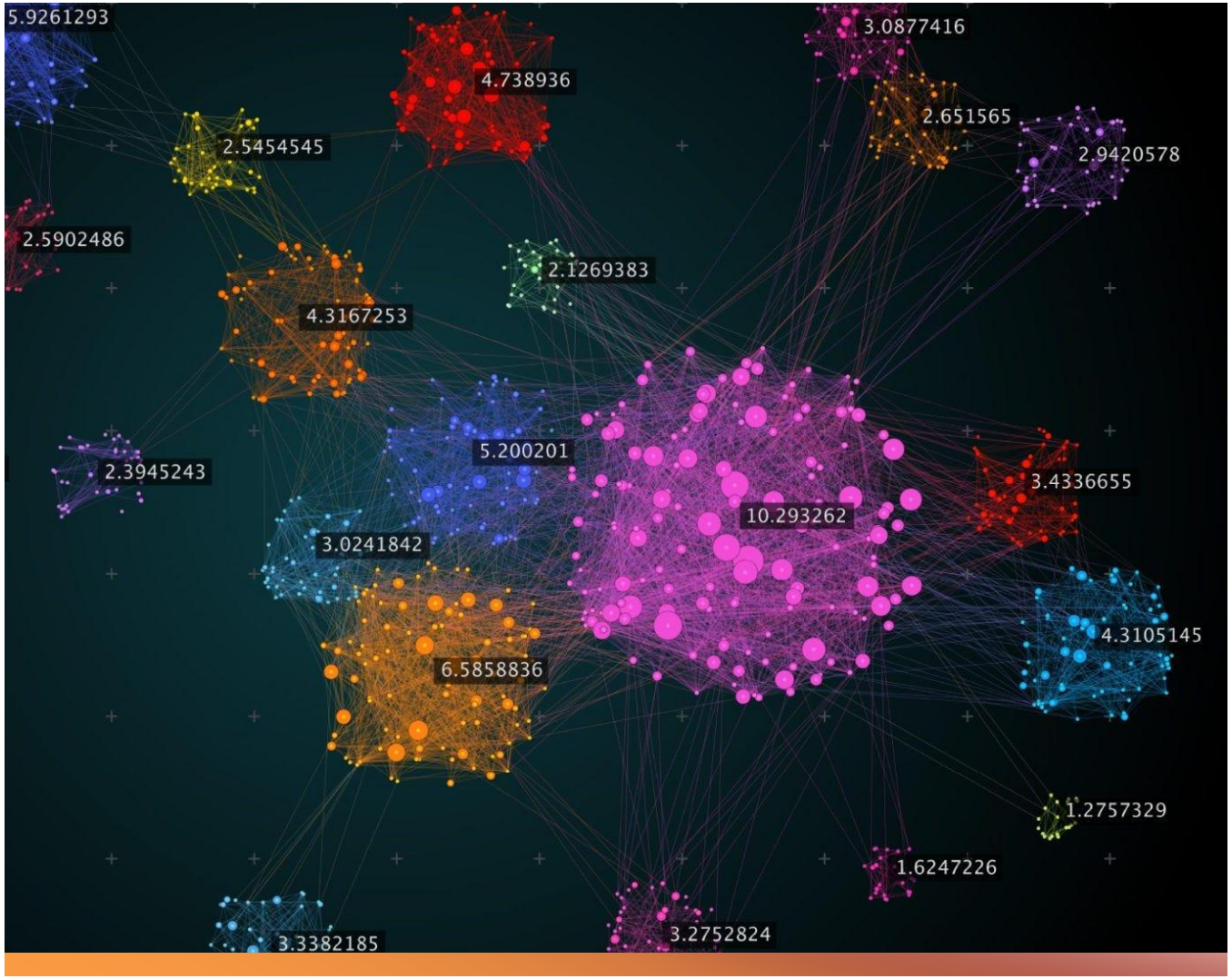

**Figure:** Disconnected Graph

# Product Network Construction




- A product network is formally modeled as a graph G = (V, E), where:
  - V represents the set of products (nodes)
  - E represents the set of edges (co-occurrence relationships)
- An edge between two nodes indicates that the corresponding products are purchased together within the same transaction.
- The strength of each edge is quantified using association metrics, such as:
  - Support → frequency of co-occurrence
  - Confidence → conditional probability
  - Lift → strength relative to independence
- Real-world product networks are often:
  - Large-scale (thousands of nodes)
  - Highly dense (hundreds of thousands of edges)
- A large portion of edges represent weak or infrequent interactions, contributing to network complexity.
- This representation captures multi-dimensional relationships across all products, which are not accessible through rule-based methods.

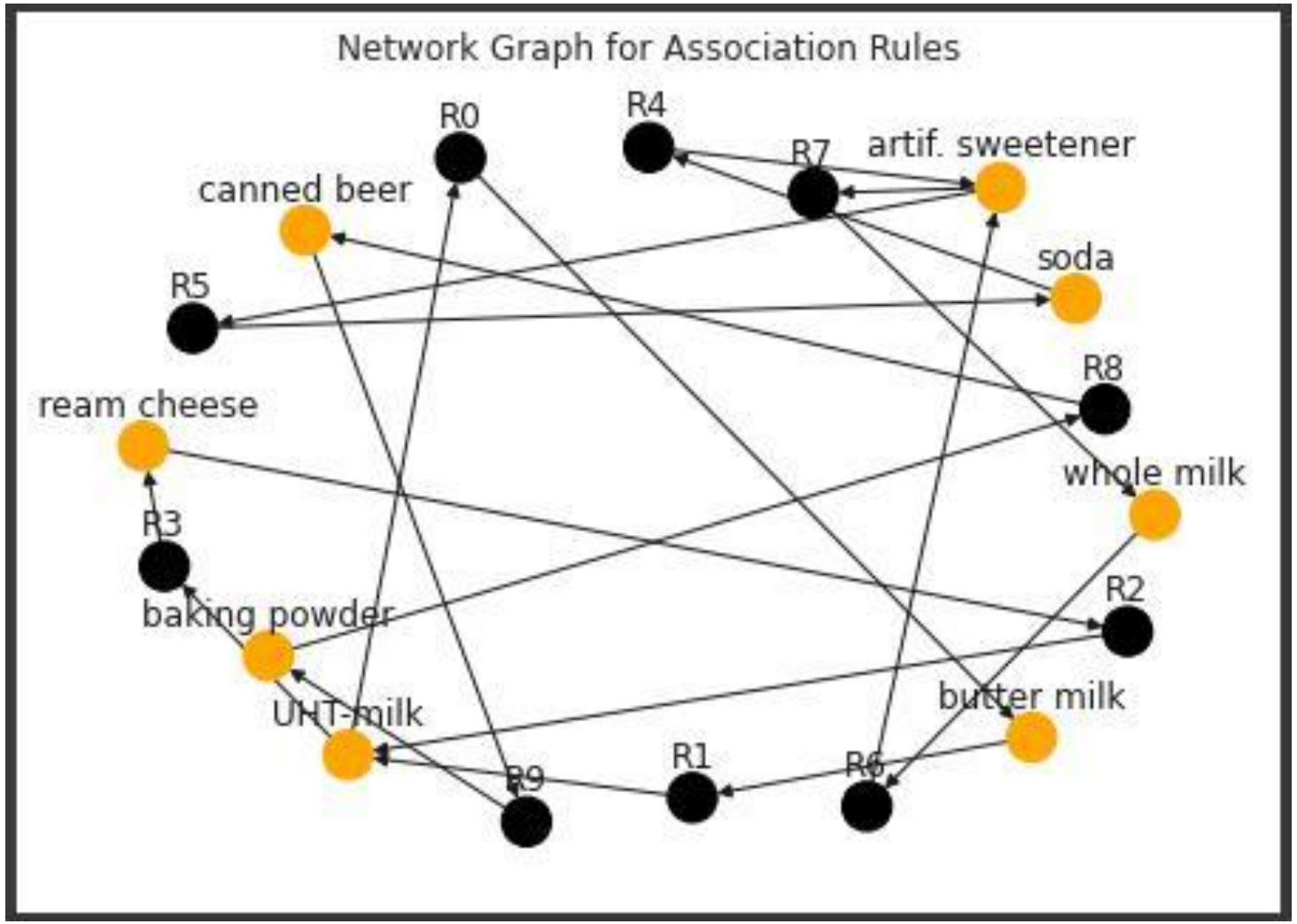
# Interpreting Product Networks



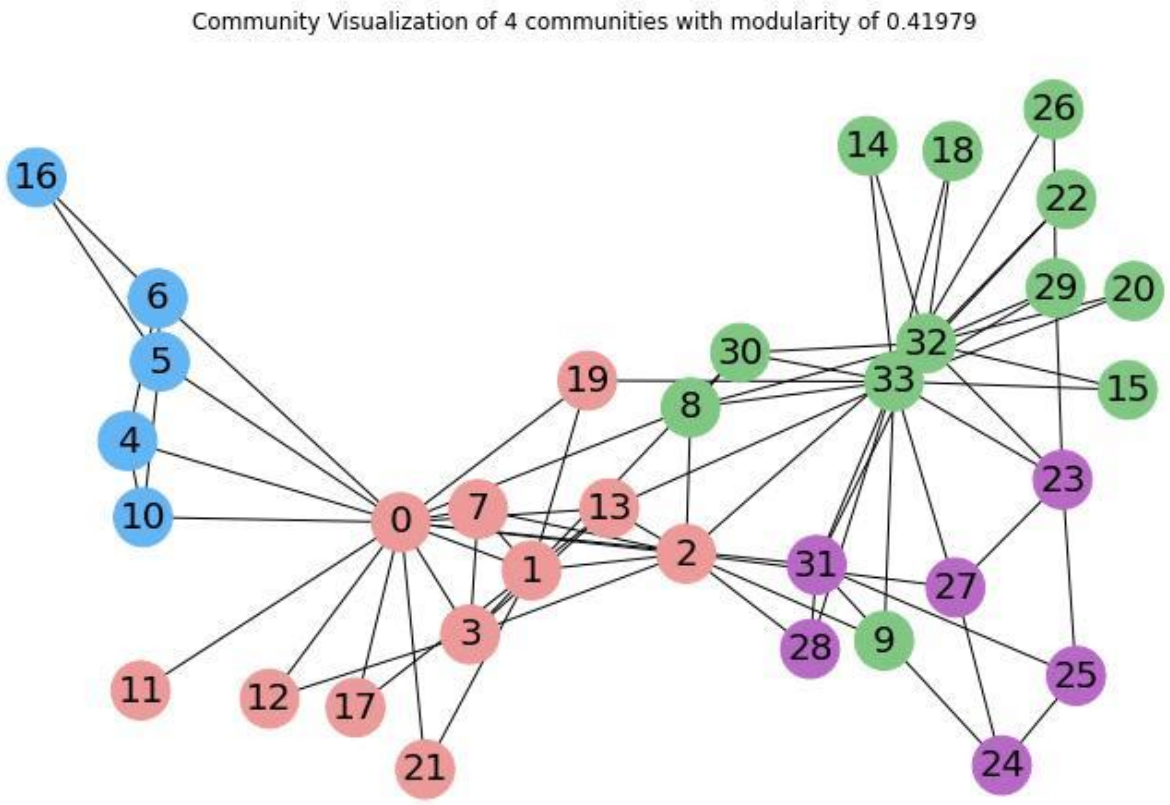

- Dense regions of the network indicate frequent co-purchase patterns, suggesting strong relationships between products.
- Sparse regions represent weak or infrequent associations, which may not be meaningful for decision-making.
- Highly connected nodes (high-degree nodes) often correspond to popular or frequently purchased products.
- Network structure allows us to identify central nodes, which may play a key role in influencing purchasing behavior.
- Clusters of interconnected nodes suggest the presence of latent groupings or behavioral patterns within the dataset.
- This level of analysis enables a more holistic understanding of product relationships compared to isolated rules.

# Product Network Visualization

- Visualization transforms complex relational data into a structured and interpretable format
- Nodes represent products, while edges represent co-occurrence relationships between items
- Dense clusters indicate strong behavioral patterns and frequent co-purchases
- Visualization enables identification of hidden structures and relationships that are not visible through statistical rules alone
- Provides an intuitive way to explore high-dimensional transactional data



# Community Detection in Product Networks




- Community detection aims to identify subsets of nodes that are more densely connected internally than externally

- These communities represent groups of products that are frequently purchased together, forming natural behavioral clusters

- This approach reduces the complexity of large networks by transforming thousands of relationships into smaller, interpretable structures

- It helps filter out noise and weak associations, allowing focus on meaningful patterns

- Common approaches include:
- -Modularity-based optimization methods
- -Graph clustering algorithms
- -Community detection provides a bridge between network analysis and unsupervised learning techniques such as clustering

# Example of Product Communities

- In a retail dataset, products naturally form communities based on purchasing behavior

- Example groupings:
- -Bread, milk, eggs → Breakfast-related products
- -Chips, soda → Snack-related products
- -Detergent, paper towels → Household essentials

- These communities represent underlying customer behavior patterns rather than isolated item relationships

- Compared to association rules, these groupings are:
- -Easier to interpret
- -More actionable for business decisions

- Applications include:
- -Store layout optimization
- -Product recommendation systems
- -Targeted marketing strategies

# What is Clustering?

- Clustering groups similar data points together
- It is a form of **unsupervised learning**
- The goal is to maximize similarity **within** clusters and difference **between** clusters
- Clusters can take different forms, not just spherical shapes
- Different clustering methods are useful for several types of data
- Clustering helps reveal hidden structure in data.

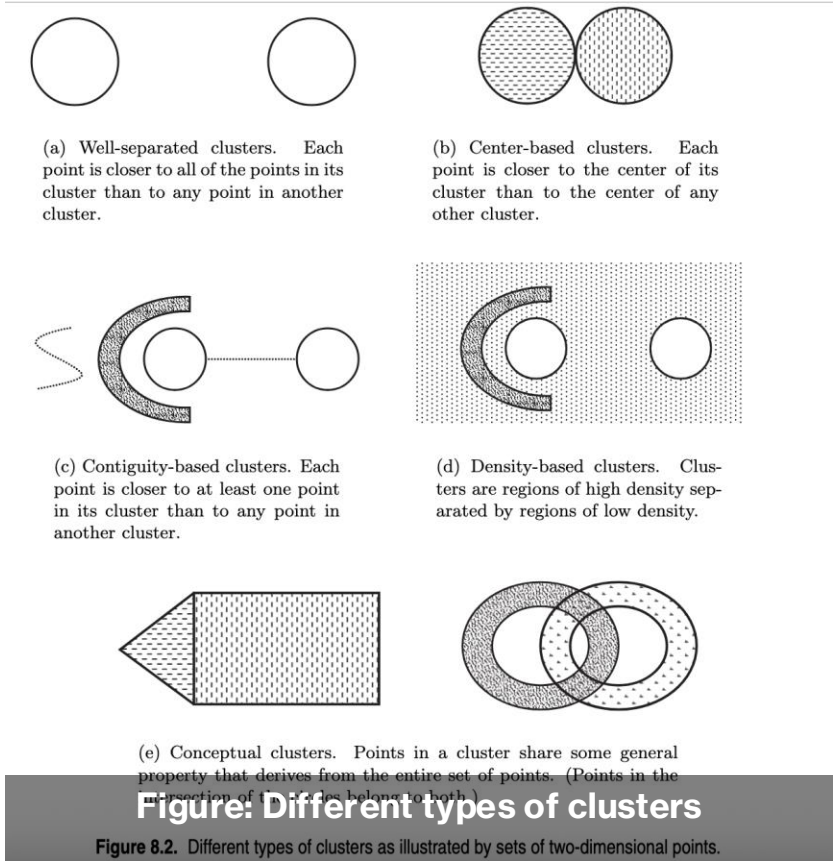



Figure: Different types of clusters

# Why Clustering Matters in Business

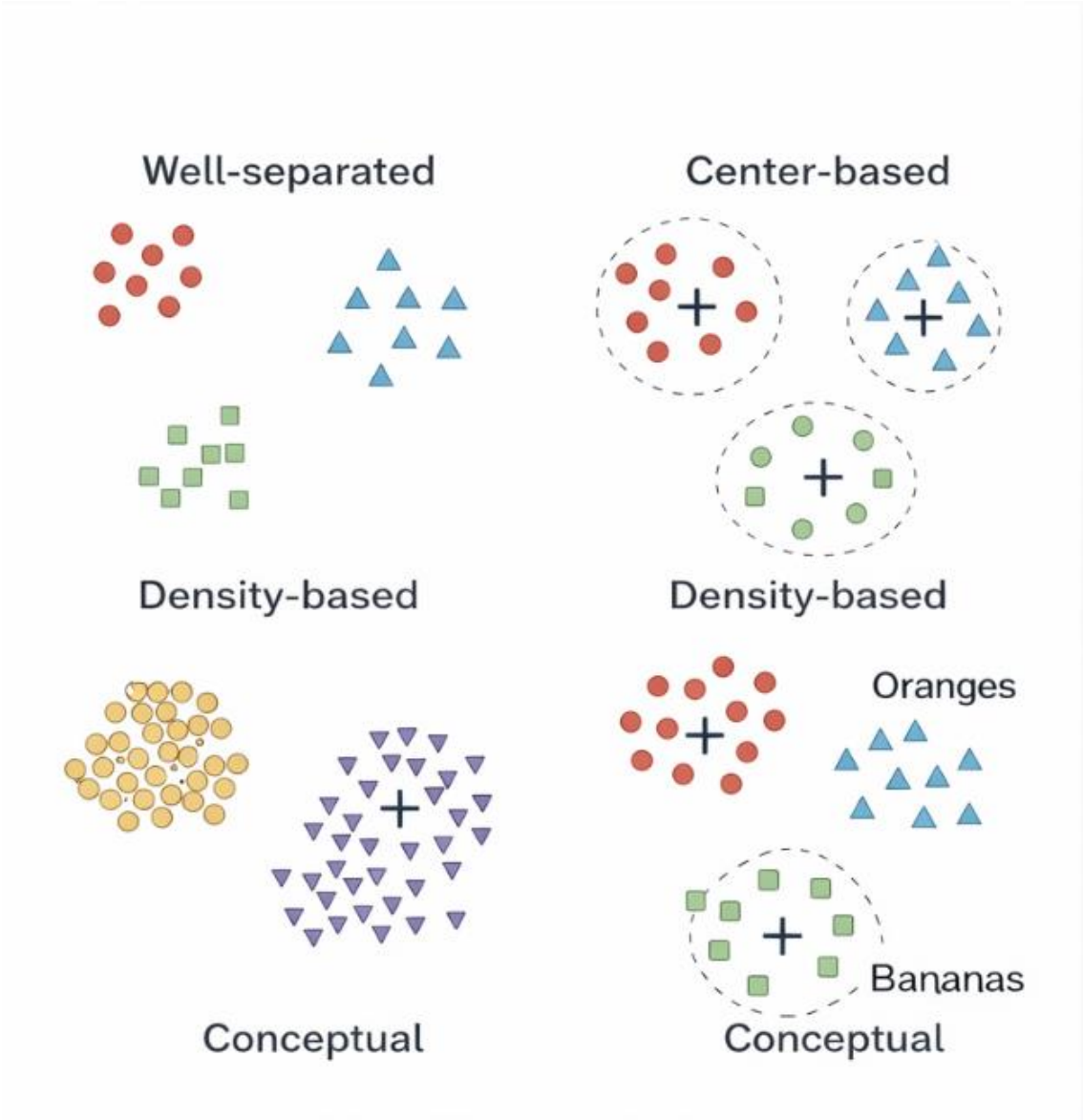
- Helps companies show **customer segments**
- Supports **targeted marketing**
- Improves **personalization**
- Makes large datasets easier to understand
- Useful in retail, finance, healthcare, and search engines

Bottom line:

**Businesses use clustering to turn raw data into actionable groups**

# Types of Clusters

- **Well-separated clusters** – groups are clearly distant from each other
- **Center-based clusters** – points are closest to a central point
- **Density-based clusters** – groups form in dense regions of data
- **Conceptual clusters** – points share a broader common property



# K-means Clustering

- **K-means** is one of the most common clustering methods
- First, choose the number of clusters, **K**
- Each data point is assigned to the **nearest centroid**
- The centroids are then **recalculated**
- The process repeats until the clusters **stabilize**

**Goal:** make each cluster as internally similar as possible

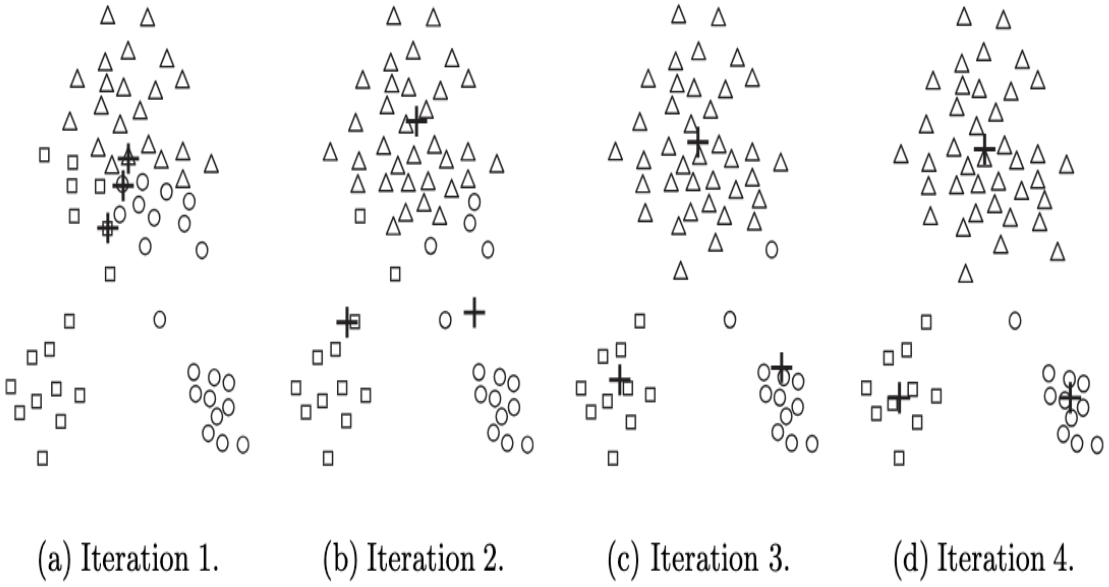



**Figure**: K-means clustering through multiple iterations

$$\min \sum_{k=1}^{K} \sum_{x_i \in C_k} ||x_i - \mu_k||^2$$

# K-means: Strengths and Limitations

**Strengths**

- Simple and easy to understand
- Efficient for large datasets
- Works well for **compact, well-separated clusters**
- Commonly used for **customer segmentation** and pattern discovery

**Limitations**

- Must choose the number of clusters (**K**) in advance
- Sensitive to **initial centroid placement**
- Struggles with:
    - **non-spherical shapes**
    - **different cluster sizes**
    - **different densities**
    - **outliers**

**Takeaway:** K-means works best when clusters are clearly separated and similar in shape.

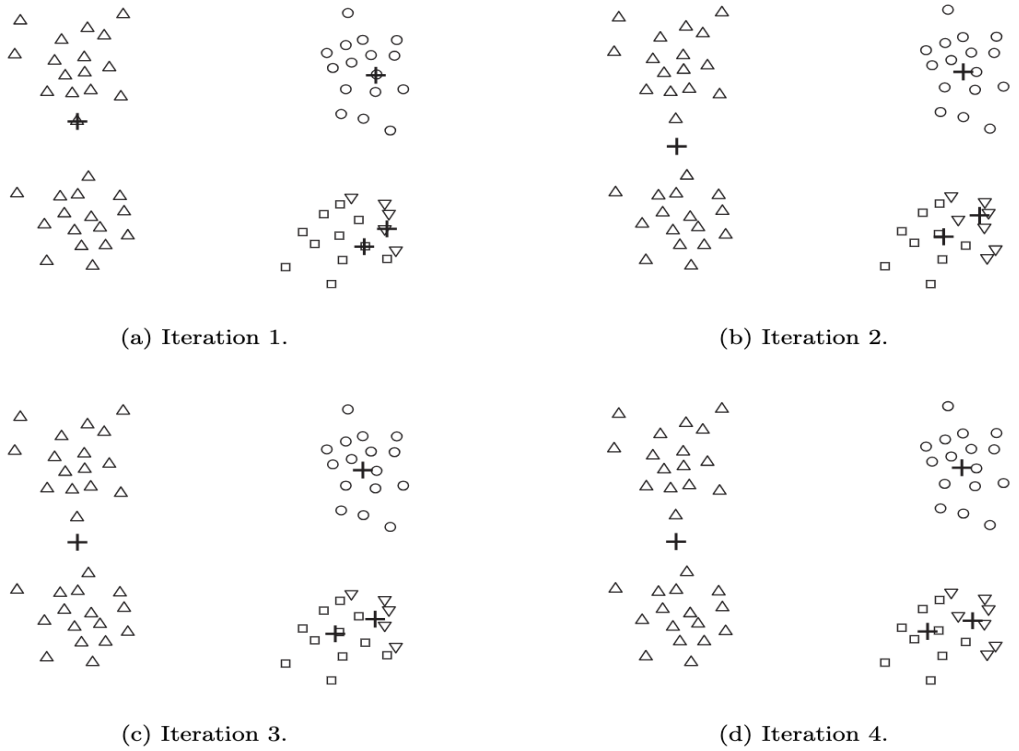



**Figure: Poor initial centroids can weaken K-means results**

# Applying Clustering to Market Basket Analysis

- Clustering helps group similar products, customers, or transactions
- Instead of looking at thousands of individual rules, clustering helps reveal larger patterns
- Products bought in similar ways can be grouped into the same cluster
- Customers with similar shopping behavior can also be segmented
- This makes market basket results easier to interpret and act on

Bottom takeaway:
Clustering turns complex transaction data into meaningful groups that support better business decisions

# Example of Clustering in a Retail Setting

- A retailer may find groups of products that appear in similar baskets
- Example product clusters:
  - **Snack cluster:** chips, salsa, dip
  - **Breakfast cluster:** cereal, milk, bread
  - **Household cluster:** detergent, paper towels, soap
- Customer clusters may also emerge, such as:
  - **Budget shoppers**
  - **Frequent bulk buyers**
  - **Convenience shoppers**
- These clusters can support **promotions, store layout, and personalized recommendations**

Bottom takeaway:

Clustering helps businesses move from isolated item rules to clearer customer and product segments.

# Clustering and Market Basket Analysis

**Market Basket Analysis**

- Finds items that are frequently bought together
- Focuses on **associations** and co-occurrence
- Often produces many rules that can be hard to interpret

**Clustering**

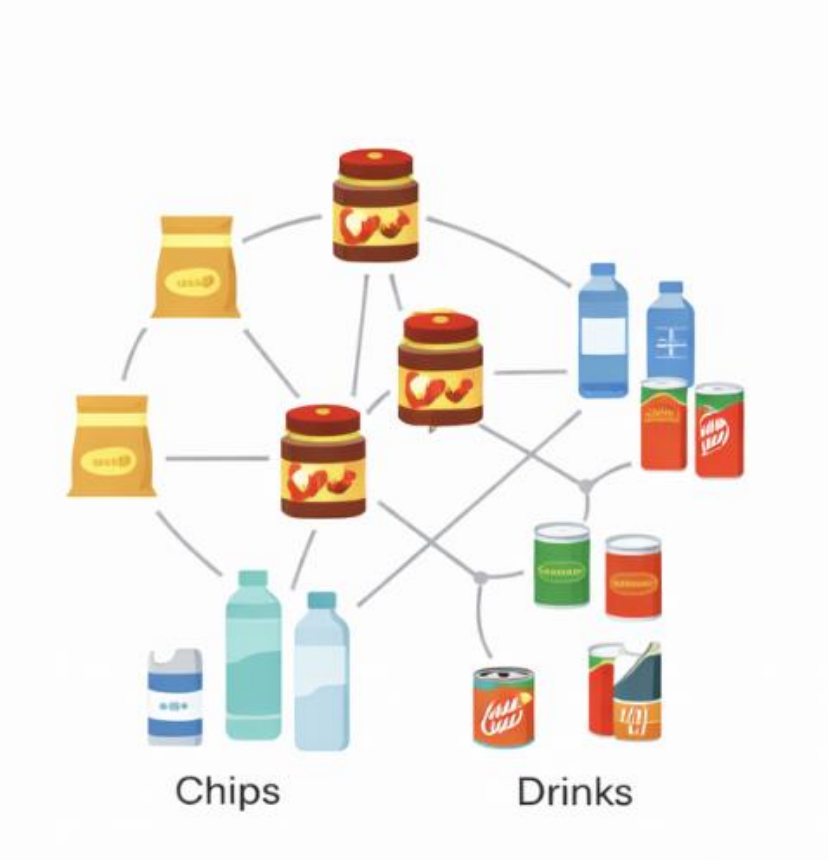
- Groups similar customers, products, or behaviors
- Focuses on **patterns** and segmentation
- Helps simplify large datasets into meaningful groups

**Together**

- Both help businesses understand customer behavior
- Clustering can make transaction data easier to interpret
- They can be used together for stronger insights

Bottom takeaway:

**Market Basket Analysis shows what is bought together, while clustering shows how data can be grouped.**



# Key Takeaways

### MBA is flexible

- The 'basket' is a metaphor — retail receipts, genomics, telecom logs.
- Same metrics. Same patterns. Infinite applications
- Rule-based, network-based, clustering – each answers a different question.

### Networks overcome rule explosion

- Raeder's community detection produces compact, expressive insights that individual rules cannot. The fruit-yogurt case shows how product variety hides real associations.

### Rule based Association *Why Rules Aren't Enough*

- Apriori and FP-Growth mine rules efficiently, but quantity doesn't mean quality.
- The real challenge isn't finding rules — it's knowing which ones actually matter

### Clustering in MBA

- Rules tell you item associations. Clustering tells you customer segments
- Clustering transforms market baskets into customer personas

### Future directions

- The future is hybrid models that combine structural and behavioral insights
- Explainable AI for retail analytics

Sources: Szymkowiak et al. (2018) Econometrics 22(1) · Raeder & Chawla (2011) Soc. Netw. Anal. Min. 1:97–113

# Thank you

Questions?

Mehreen Ali Gillani
Izza Khan
Sachi kapoor

Predictive Analytics - DATA624
Professor Nieman Jeffery